A Machine-Checked Case Study Motivated by Recent Agent2Agent Discussion: Cocos AI aTLS

Akira Okutomi

April 12, 2026

Abstract

Motivated by the recent IETF agent2agent discussion about how to distinguish informal security goals from machine-checked claims, this report presents a small machine-checked case study of Cocos AI aTLS [1]. In this workspace, the legacy/current comparison already shows that moving to the post-handshake design does not by itself resolve same-endpoint authenticity under the explicit leakage assumption used by the relay models. The current verifier additionally exposes canonical exporter-label acceptance and a weaker missing-attestation-after-offer issue. The workspace combines implementation regression tests, Tamarin models, and ProVerif [10, 11] models, and also includes narrower checks for leaf-key substitution and session-scoped context reuse. It therefore distinguishes informal security goals for the design from a smaller set of machine-checked claims that this workspace can already support directly.

1 Overview

This report is framed as a machine-checked case study motivated by the recent agent2agent discussion about protocol goals and machine-checked claims. Within that framing, the workspace is centered on a small set of security questions about attested origin, binding completeness, and same-endpoint authenticity. Its main result is already visible at that level: moving from the legacy pre-handshake design to the current post-handshake design does not by itself resolve same-endpoint authenticity under the explicit leakage assumption used by the relay models, while the current verifier additionally exposes canonical exporter-label acceptance and a weaker missing-attestation-after-offer issue. For authenticated AI infrastructure, there is also a separate identity question beyond channel ownership alone: an accepted peer may still need to be distinguished from another co-resident AI agent or local stand-in on the same attested machine. The current Tamarin models now expose that same-machine wrong-agent trace directly and also include a narrow intended-agent-bound mitigation sketch in which that trace disappears.

Across the workspace, Go-language regressions act as implementation anchors for concrete verifier behavior. Tamarin is used for request/offer gating, attestation presence, canonical-label enforcement, replay/one-shot behavior, and a same-endpoint attack trace under leakage. ProVerif is used for correspondence-style clarification of attested origin vs. same-endpoint authenticity, including the legacy/current comparison.

This workspace was also motivated by the IETF agent2agent mailing-list discussion around how to separate informal security goals from smaller machine-checked claims in protocol analysis [2, 3].

The current workspace is organized around a small verification tree plus a small set of implementation regressions.[1]

[1] Main repo locations: `verification/` and `pkg/atls/ea/authenticator_test.go`.

This report should not be read as a condemnation of Cocos AI or of the broader goal of authenticated AI infrastructure. The project is attempting ambitious work at the edge of current protocol and attestation practice, and that is precisely why early machine-checked scrutiny is useful here. The intent of this workspace is to make design assumptions and boundary conditions explicit while the design is still evolving, not to dismiss the underlying direction.

2 Scope

After the legacy pre-handshake design anchored around commit `e372cfc`, Cocos AI later introduced the current post-handshake exported-authenticator form anchored around commit `80bf813` (as of 9 April 2026).

This report focuses on that current post-handshake aTLS behavior, while keeping a compact legacy/current design split and a minimal reproducibility path for the older pre-handshake design. On the current side, it examines relay/diversion, canonical exporter-label enforcement, missing attestation after offer, leaf-key substitution, and context-reuse questions through a mix of implementation tests and small formal models.

More concretely, it focuses on the exported-authenticator[2] path used around Cocos AI aTLS:

1. the client completes a TLS 1.3 handshake,
2. the client sends an exported-authenticator request with a fresh request context,
3. the server constructs an authenticator and may attach CMW attestation[3] data as a leaf extension,
4. the client validates the authenticator and then verifies the attestation binder against the connection state, request context, and leaf public key.

The modeling style is deliberately minimal. Each artifact aims to be a small working model for one question at a time, rather than a single monolithic proof attempt for the entire protocol stack.

3 Assumptions

- The relay/diversion analysis uses an explicit leakage assumption: the attested ephemeral private key is revealed after attestation issuance.
- The context-reuse result is session-scoped: with `ea.Session`[4] the workspace checks a one-shot guarantee; without session tracking, replay remains possible at the API level.

4 Out of Scope

- A full symbolic model of TLS 1.3 or of the complete Cocos AI implementation.
- End-to-end exploitability of the missing-attestation-after-offer result under the default server wiring.
- Full X.509 certificate-chain validation and complete evidence-format parsing.

[2] In this report, "exported authenticator" refers to the post-handshake TLS authenticator exchange used by the current Cocos AI design, rather than a certificate message carried inside the initial handshake [5].

[3] In this report, "CMW attestation" refers to the attestation payload and certificate-extension format used by the current Cocos AI aTLS implementation.

[4] In this report, `ea.Session` refers to the verifier-side state used to remember which request contexts have already been accepted, so that the same context cannot be accepted twice.

5 Design Split: Legacy vs. Current

Cocos AI aTLS should be read as two different designs rather than one continuous protocol:

- **Legacy pre-handshake aTLS.** The older design used OpenSSL custom extensions during the TLS 1.3 handshake. In that design, an evidence request was sent in `ClientHello`, a TEE type value was carried in `EncryptedExtensions`, and the attestation report was attached as a certificate extension.
- **Current post-handshake aTLS.** The current design completes the TLS 1.3 handshake first and then performs a post-handshake exported-authenticator exchange that carries attestation data in the authenticator response.

This report focuses primarily on the current post-handshake design, but it also keeps a minimal reproducibility path for the legacy pre-handshake design so that the two generations are not conflated.

The immediate design lesson from comparing the two generations is narrow but important. Moving attestation from the handshake to a post-handshake exported-authenticator exchange changes message placement and some binding inputs, but it does not by itself establish same-endpoint authenticity. On the relay question modeled in this report, the key requirement is not only where attestation is carried, but whether the accepted peer is demonstrably tied to the client's exact live channel.

Comparison point	**Legacy pre-handshake**	**Current post-handshake**
Attestation placement	Carried inside the TLS handshake.	Carried in a post-handshake exported authenticator.
Main binding in this workspace	Client nonce + leaf public key.	Request context + leaf public key + exported TLS value.
Checked here	Request gating and nonce/public-key binding.	Canonical exporter-label enforcement, missing attestation after offer, leaf-key substitution, context reuse, and relay / same-endpoint authenticity under leakage.
Additional note	Fail-closed `NO_TEE` is checked as a narrower implementation condition.	The current generation is the main subject of the report and has broader reproducibility coverage.

Table 1: Comparison of the two Cocos AI aTLS generations. The key takeaway for this report is that message placement changes between the two generations, but same-endpoint authenticity still requires an explicit binding to the client's exact live channel.

The legacy side is therefore kept mainly as a comparison point and as a compact methodological example. In this workspace it confirms acceptance only after a prior request and nonce/public-key binding, while also checking fail-closed `NO_TEE`[5] handling as a narrower implementation condition. A more detailed sketch of the legacy flow is left to the Appendix.

6 Verification Artifacts and Implementation Flow

The workspace is organized around three narrowly scoped security questions:

1. whether client acceptance preserves origin authenticity at the attestation level,
2. what is explicitly bound into the attestation and binder inputs, especially the request context, the leaf public key, and any TLS exporter-derived values, and

[5] Here `NO_TEE` means that no TEE is selected or attested for the connection.

3. whether client acceptance further establishes same-endpoint authenticity, rather than only attested origin authenticity plus proof of possession of the attested leaf key.

The detailed code-path inventory and artifact layout are kept in the verification README.[6]

6.1 Current Post-Handshake Flow

At the implementation level, the current Cocos AI flow relevant to this report is:

1. the client first completes a TLS 1.3 handshake,
2. the client builds an exported-authenticator request containing a fresh context and the attestation offer extension,
3. the server receives that request and calls `BuildLeafExtensions(...)` to construct the leaf certificate-entry extensions for the authenticator response,
4. the default provider computes a binding from `ExportKeyingMaterial(label, context)` and the leaf public key, then embeds evidence plus binder values into a CMW attestation extension,
5. the client verifies the authenticator and, when the extension is present, parses the attestation payload and verifies the binder.

This is the execution path that the current verification artifacts try to mirror.

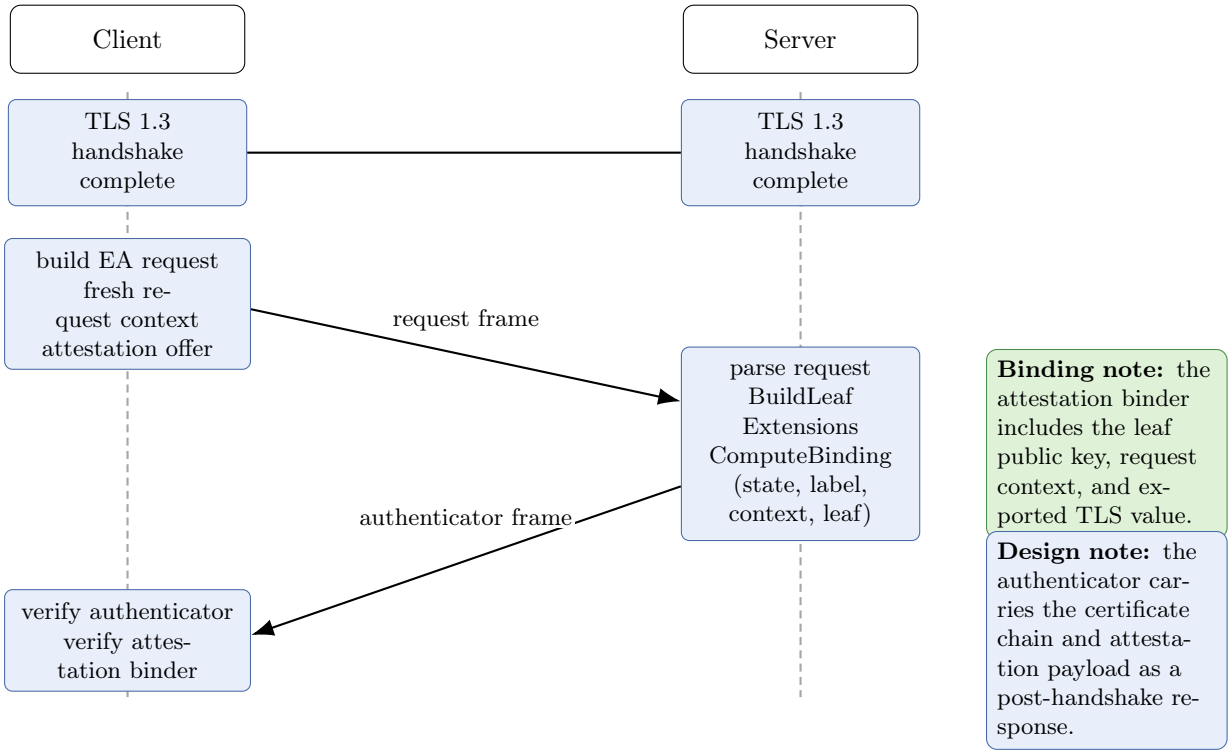

Figure 1: Current post-handshake exported-authenticator flow. Blue boxes show protocol steps; green notes show values explicitly bound into the attestation binder.

[6] `verification/README.md`. For the current design, the main moving parts are the internal transport request/response path, the attestation provider, and the binder/verifier helpers under `pkg/atls/`.

7 Main Findings Summary

Finding	Evidence	Result	Interpretation
Redesign effect on same-endpoint authenticity	legacy ProVerif + current ProVerif + current Tamarin[7]	not resolved by the post-handshake redesign under the explicit leakage assumption	message placement changed, but live-channel binding remains unestablished
Relay/diversion risk reported in CVE-2026-33697 [6, 7, 8]	current ProVerif + current Tamarin attack trace	accepted origin persists, but same-endpoint authenticity fails under leakage	CVE-aligned design-level concern, not a full exploit proof
Same-machine intended-agent identity	current Tamarin + intended-agent-bound mitigation sketch	machine-level attestation alone does not distinguish the intended agent; the mitigation sketch removes that trace in this abstraction	attested machine identity is still not intended-agent identity
Canonical exporter-label enforcement	Go regression + falsified Tamarin lemma + ProVerif correspondence	canonical-label enforcement fails in current verifier behavior	strongest additional finding in this workspace
Missing attestation after offer	Go regression + falsified Tamarin lemma	API-level acceptance remains possible after an explicit offer	weaker; end-to-end exploitability is not established here

Table 2: Summary of the main findings supported by this workspace.

Go-language regressions, Tamarin models, and ProVerif models play different roles here rather than repeating the same check in three forms. The Go-language tests anchor concrete verifier behavior in the current code path. Tamarin captures gating, attestation presence, canonical-label enforcement, replay/one-shot behavior, and a same-endpoint attack trace under leakage. ProVerif clarifies correspondence-style questions about attested origin, same-endpoint authenticity, and the legacy/current comparison. A wider coverage inventory is included in the Appendix.

8 Current Design Results

8.1 Implementation Regression Tests (Go-language)

The current-design Go regressions live in the main authenticator test file.[8] Their role here is not to provide a formal security proof, but to act as implementation anchors for several narrow security-relevant behaviors in the current code path. In the report, the important point is what those tests check rather than their exact names. The current set covers four behaviors:

- acceptance of a non-canonical exporter label during attestation verification,

[7] The initial Tamarin model in this workspace did not target the relay/diversion assumptions behind CVE-2026-33697. After the models were extended toward same-endpoint authenticity under an explicit ephemeral-key leakage assumption, Tamarin also exposed related behavior.

[8] `pkg/atls/ea/authenticator_test.go`

- API-level acceptance when an explicit attestation offer is followed by an authenticator without an attestation extension,
- a narrow positive check against leaf-key substitution, and
- the split between session-scoped one-shot context handling and replayable API-level validation without session tracking.

The exact test names and reproduction commands are documented in the verification README.[9]

8.2 Tamarin Results

The current-design Tamarin artifacts are summarized in the Tamarin README.[10] Together they cover attestation offer vs. plain request, authenticator origin, attestation presence, canonical exporter-label enforcement, and one-shot vs. replayable request-context usage.

Property	**Meaning**	**Result**
(1) Server-origin authenticity	An attested client acceptance must correspond to a server-side authenticator creation event.	verified
(2) Offer-gated acceptance	A successful acceptance must be preceded by a client attestation offer.	verified
(3) No attested acceptance from plain mode	Plain requests do not lead to attested acceptance.	verified
(4) Canonical exporter-label enforcement	Accepted attestations should only use the canonical exporter label.	falsified
(5) No success without attestation after offer	An offered request should not succeed when the attestation extension is missing.	falsified
(6) Same-endpoint failure under leakage	Under the explicit leakage assumption, there exists a client-accepted session that does not establish same-endpoint authenticity.	trace found
(7) Session-scoped one-shot context use	With explicit session tracking, the same request context cannot be accepted twice.	verified
(8) Replay without session tracking	Without session tracking, there exists a replay trace for the same request context.	trace found
(9) Wrong-agent trace on the same machine	A client acceptance of the intended agent can still be satisfied by another co-resident agent on the same attested machine.	trace found
(10) Wrong-agent trace under intended-agent-bound mitigation	Under the intended-agent-bound sketch, that same-machine wrong-agent trace is no longer found.	no trace found

Table 3: Summary of current-design Tamarin results across the current models. File mapping under `verification/tamarin/`: (1)–(5) `cocos_attestation.spthy`; (6) `cocos_same_endpoint_current.spthy`; (7)–(8) `cocos_context_reuse_current.spthy`; (9) `cocos_agent_identity_current.spthy`; (10) `cocos_agent_identity_bound_current.spthy`.

[9] `verification/README.md`
[10] `verification/tamarin/README.md`

The falsified canonical exporter-label lemma is the stronger additional finding. The falsified missing-attestation-after-offer lemma is weaker because its reachability in the default server wiring has not been shown end-to-end.

A separate current-design Tamarin model checks same-endpoint authenticity under the explicit leakage assumption.[11] In that model:

- `received_attestation_has_server_origin` is verified,
- `same_endpoint_can_fail_under_leakage` has a verified attack trace.

This mirrors the relay/same-endpoint split also seen in the ProVerif artifacts.

A separate session-scoped Tamarin model checks a different question.[12] In that model, `session_context_is_one_shot` is verified and `no_session_replay_exists` has a verified replay trace. This is meant to track the semantics of `ea.Session`; it should not be read as a claim that every current-design validation path is automatically one-shot.

A second pair of current-design Tamarin models checks a different identity gap: whether machine-level attestation can distinguish the intended AI agent from another co-resident agent on the same attested machine.[13] The unbound model verifies machine origin and yields a wrong-agent attack trace on the same machine. The mitigation sketch also verifies machine origin, but the same wrong-agent trace is no longer found. This remains a narrow design sketch, but in this abstraction it shows that binding both the attestation and the live response to the intended agent removes the co-resident-agent confusion trace.

8.3 ProVerif Results

The ProVerif artifacts are summarized in the ProVerif README.[14] They were added because the first Tamarin model, which was not explicitly focused on the CVE-aligned relay/diversion question, did not cover ephemeral key compromise, distinct channels or sessions, or a correspondence goal for same-channel endpoint authenticity.

The current side carries the main ProVerif weight in this report. The legacy-side ProVerif results are retained in the Appendix only as a compact comparison point; the main body keeps the emphasis on what the current post-handshake design does and does not establish.

The attack assumption behind the published relay-risk discussion is represented explicitly by leakage of the ephemeral private key after attestation issuance.

[11] `verification/tamarin/cocos_same_endpoint_current.spthy`

[12] `verification/tamarin/cocos_context_reuse_current.spthy`

[13] `verification/tamarin/cocos_agent_identity_current.spthy`; `verification/tamarin/cocos_agent_identity_bound_current.spthy`

[14] `verification/proverif/README.md`

Current-Design Results

Model thread	Correspondence query	Result	Interpretation
(1) Request-gated acceptance	`ClientAccepts ==> ClientSendsEARequest`	true	Acceptance still implies a prior client-side exported-authenticator request.
(2) Server-side attestation origin	`ClientAccepts ==> ServerIssuesAttestation`	true	Client acceptance still tracks genuine server-side attestation origin.
(3) Server-side binding/construction event	`ClientAccepts ==> ServerBuildsAuthenticator`	false	Under the explicit leakage assumption, the current relay model does not preserve the same server-side channel/build event for an accepted client session.
(4) Relay / same-endpoint authenticity	`current: ClientAccepts ==> ServerBindsSameChannel`	false	Under the explicit leakage assumption, the current relay model does not establish same-endpoint authenticity for a client-accepted session.
(5) Canonical exporter-label enforcement	`current: ClientAccepts ==> ServerUsesCanonicalLabel`	false	Acceptance does not imply that the canonical exporter label was used.
(6) Leaf-key substitution resistance	`current: ClientAccepts ==> ServerAttestsLeafKey`	true	Within this abstraction, client acceptance remains tied to the attested leaf public key.

Table 4: Summary of the current-design ProVerif correspondence queries used in this workspace. File mapping under `verification/proverif/`: (1)–(4) `cocos_relay_attack.pv`; (5) `cocos_exporter_label_current.pv`; (6) `cocos_leaf_key_substitution_current.pv`.

Taken together with the other checks, the ProVerif side gives a narrower binding picture for the current design: it supports binding to the request context and leaf key, but not to the client's live peer identity or same-endpoint authenticity under the explicit leakage assumption used by the relay models. Read together with the Appendix, this also shows that moving from the legacy pre-handshake design to the current post-handshake design does not by itself resolve the same-endpoint problem.

Figure 2 gives the relay/diversion intuition behind those current-design ProVerif results and highlights the gap between attested origin and same-endpoint authenticity.

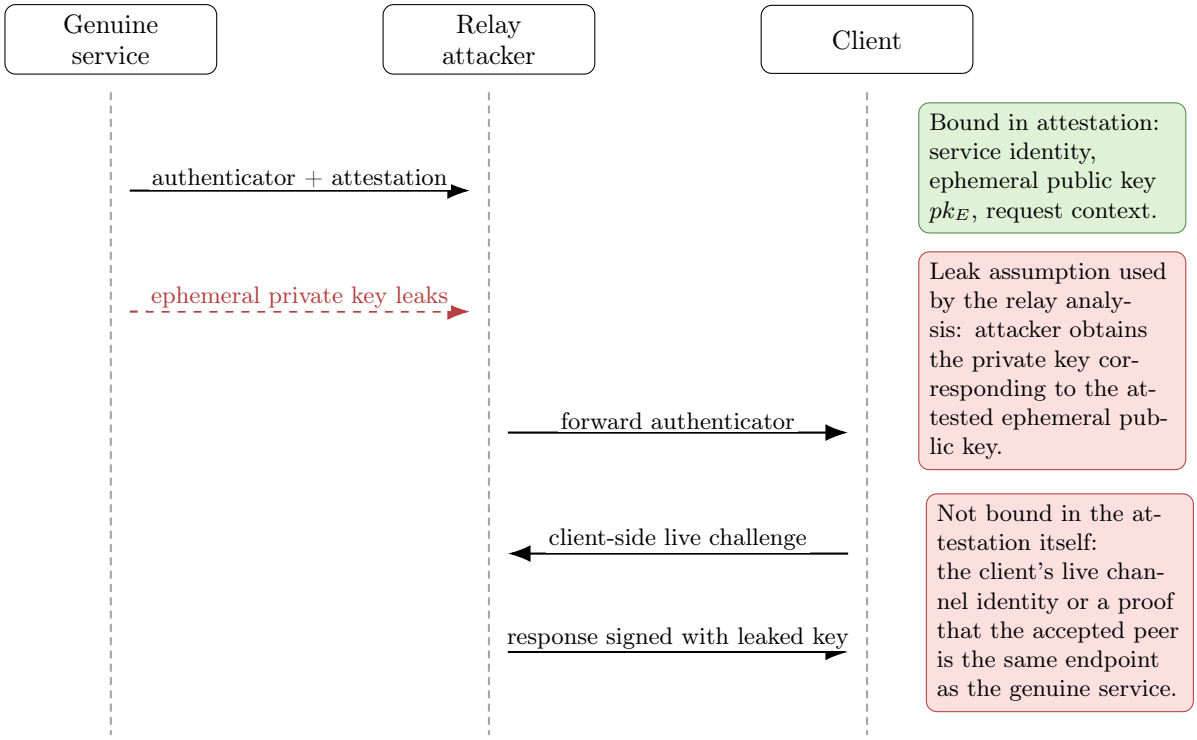

Figure 2: Relay/diversion intuition aligned with the risk reported in CVE-2026-33697. Green notes mark values explicitly bound into the attestation binder; red notes mark the leakage assumption or values not bound to the client's live channel proof.

9 Limits and Interpretation

Taken together, the workspace narrows several important questions to machine-checked results, but it does not elevate every design goal into a proved theorem.

9.1 Relay / same-endpoint authenticity under leakage

The relay/diversion and same-endpoint authenticity results are design-level and depend on the explicit ephemeral-key leakage assumption used by the ProVerif and Tamarin models. They are best read as evidence aligned with the relay/diversion risk reported in CVE-2026-33697 [6, 7, 8], not as a full end-to-end exploit proof for the complete implementation.

Within this workspace, the move from the legacy pre-handshake design to the current post-handshake design does not eliminate the same-endpoint authenticity failure under that explicit leakage assumption. In that narrow CVE-aligned sense, the redesign by itself does not yet provide evidence that the issue has been resolved.

For authenticated AI infrastructure, same-endpoint authenticity is not an optional refinement: it is needed to distinguish the genuine remote endpoint from a relay or stand-in that can present valid attestation material. That requires more than a redesign of message placement alone. At minimum, the protocol needs a proof that the accepted peer is bound to the exact live channel seen by the client, rather than only to attested origin plus possession of the attested key. Under the explicit leakage assumption used here, that likely requires either a stronger non-exportability assumption for the attested key material or an additional non-transferable channel-bound proof that a relay attacker cannot replay from a different endpoint. Even that would still leave a separate AI-agent identity question. In a multi-agent machine, proving the exact live channel is not yet the same thing as proving that the accepted peer is the intended agent rather than another co-resident agent or local stand-in using the same attested machine.

Even same-endpoint authenticity would still not by itself establish intended agent identity or role authorization.

9.2 Same-machine intended-agent identity

The new Tamarin agent-identity models make that AI-specific gap concrete. In the unbound machine-level model, a client can target one intended agent and still accept a response produced by another co-resident agent on the same attested machine. This means that machine identity alone is not yet intended agent identity.

The mitigation sketch is deliberately narrow, but still useful. In that model, the attestation explicitly names the intended agent and the live response is accepted only in an intended-agent-bound form. Under that abstraction, the direct wrong-agent trace disappears. This should not yet be read as a complete deployment recipe, but it is a positive signal that agent-bound attestation plus agent-bound live proof is a plausible direction for closing the same-machine stand-in risk.

9.3 Canonical Exporter-Label Enforcement

The canonical exporter-label issue is the strongest additional finding in this workspace because it is supported by three separate anchors: a falsified Tamarin property, a ProVerif correspondence whose canonical-label condition fails, and a Go regression showing acceptance of an alternate exporter label.

9.4 Missing Attestation After Offer

The missing-attestation-after-offer issue is weaker. The workspace supports it with a falsified Tamarin property and a Go regression, but the result remains API-level. The default server wiring appears more fail-closed than this verifier behavior alone suggests, so end-to-end exploitability is not established here.

9.5 Leaf-key substitution and context reuse

The workspace also includes two narrower results. First, leaf-key substitution is supported by a Go regression and by a ProVerif correspondence in which `ClientAccepts ==> ServerAttestsLeafKey` holds. Second, request-context one-shot behavior is established only for the explicit `ea.Session` path; without session tracking, replay remains possible at the API level. In that sense, the current design shows a scoped one-shot guarantee rather than a general replay-resistant result.

10 Reproducibility and Repository

This workspace is based on the Cocos AI repository [1], with the legacy/current discussion anchored around commits `e372cfc` and `80bf813`, respectively.

The current design is reproduced through the repository wrapper scripts.[15] The recommended order is legacy first, then Go tests, then Tamarin, then ProVerif.

The legacy design is kept as a compact comparison point rather than as the main subject of the report. The legacy wrapper script[16] checks out commit `e372cfc` into a temporary worktree and runs a narrow legacy regression anchor. It is paired with the legacy Tamarin model,[17] which

[15] `verification/run-legacy-tests.sh`; `verification/run-go-tests.sh`; `verification/run-tamarin.sh`; `verification/run-proverif.sh`

[16] `verification/run-legacy-tests.sh`

[17] `verification/tamarin/cocos_legacy_attestation.spthy`

verifies request-gated acceptance, fail-closed `NO_TEE` handling, and nonce/public-key binding. The legacy ProVerif comparison models[18] add matching formal results on the same theme.

The verification workspace overview, tool-specific notes, and relay-aligned evidence note are all linked from the repository documentation.[19]

Raw command outputs, scripts, and file inventories are intentionally kept in the repository rather than repeated in the PDF.

11 Open Problems

This workspace narrows several questions to machine-checked results, but it also leaves a short list of concrete follow-up checks:

1. how to guarantee same-endpoint authenticity even under the explicit leakage assumption used by the relay models,
2. how to distinguish the intended remote AI agent from another
3. how to distinguish the intended remote AI agent from another co-resident agent or local stand-in on the same attested machine, and whether the intended-agent-bound mitigation sketch can be turned into a stronger protocol guarantee,
4. whether canonical exporter-label enforcement can be made fail-closed in the verifier,
5. whether missing attestation after an explicit offer is blocked end-to-end in the default server wiring,
6. how much replay resistance can be guaranteed outside the explicit `ea.Session` path,
7. and whether extension-placement / certificate-entry confusion should be checked more broadly beyond the current path already covered here.

12 Conclusion

Early machine-checked scrutiny is useful here because it already exposes meaningful protocol results early in the design cycle. In particular, the legacy/current comparison shows that relocating attestation into the post-handshake exported-authenticator path does not by itself resolve same-endpoint authenticity on the relay question modeled here. The current verifier path also exposes additional problems beyond that published relay-aligned concern, most clearly in canonical exporter-label enforcement and more weakly in missing attestation after offer. More broadly, attested machine identity is not yet the same thing as intended AI-agent identity on that machine, and same-endpoint authenticity would still not by itself establish intended-agent identity or role authorization. The new Tamarin agent-identity models also suggest that machine-level attestation alone leaves room for a same-machine wrong-agent stand-in, while an intended-agent-bound sketch can remove that trace in this abstraction. Together with the narrower leaf-key and session-scoped replay results, this makes the workspace a scoped, reproducible starting point for further checking rather than a complete endpoint-verification story.

[18] `verification/proverif/cocos_legacy_request_binding.pv`; `verification/proverif/cocos_legacy_relay_attack.pv`

[19] Overview: `verification/README.md`. Tamarin notes: `verification/tamarin/README.md`. ProVerif notes: `verification/proverif/README.md`. Relay note: `verification/proverif/CVE-2026-33697-evidence.md`.

References

References

[1] Cocos AI repository used by this workspace: https://github.com/ultravioletrs/cocos.

[2] IETF agent2agent mailing-list discussion, "Re: Question about `draft-howard-virp-03`," including the clarification that the draft did not yet carry machine-checked proof claims and the suggestion to frame such statements as informal security goals.

[3] M. U. Sardar, "TLS FATT Extension," Internet-Draft, https://www.ietf.org/archive/id/draft-usama-tls-fatt-extension-02.html#section-4.

[4] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, 2018.

[5] K. Bhargavan, B. Delignat-Lavaud, and others, "Exported Authenticators in TLS," RFC 9261, 2022.

[6] GitHub Security Advisory, "GHSA-vfgg-mvxx-mgg7: CoCoS attested TLS is vulnerable to relay attacks via extracted ephemeral TLS keys," https://github.com/ultravioletrs/cocos/security/advisories/GHSA-vfgg-mvxx-mgg7.

[7] NVD entry for CVE-2026-33697: https://nvd.nist.gov/vuln/detail/CVE-2026-33697.

[8] OSV entry for CVE-2026-33697: https://osv.dev/vulnerability/CVE-2026-33697.

[9] Public discussion around CVE-2026-33697, plus a local repository note that summarizes how the workspace aligns its relay/diversion models with that discussion.

[10] D. Basin, C. Cremers, J. Dreier, S. Meier, R. Sasse, and B. Schmidt, *Tamarin Prover*.

[11] B. Blanchet, V. Cheval, and M. Sylvestre, *ProVerif*.

Appendix

Workspace Coverage Matrix

Attack family	Legacy pre-handshake	Current post-handshake
Freshness-token / request-context binding	`checked`	`checked`
Context reuse / one-shot request contexts	`not yet`	`partial`
Certificate / public-key substitution	`checked`	`checked`
Legacy `NO_TEE` fail-closed handling	`checked`	`n/a`
Current missing attestation after explicit offer	`n/a`	`checked`
Extension placement / certificate-entry confusion	`not yet`	`checked`
Canonical exporter-label enforcement	`n/a`	`checked`
Relay / same-endpoint authenticity under leakage	`checked`	`checked`
Same-machine intended-agent identity	`n/a`	`checked`

Table 5: Workspace coverage across legacy and current Cocos AI aTLS designs. Here, "checked" means that the question is directly exercised by a test or formal model result in this workspace; it does not imply that the answer is always positive. The strongest current-design signals are freshness-token binding, certificate/public-key substitution, missing-attestation handling, canonical exporter-label enforcement, relay / same-endpoint authenticity under the explicit leakage assumption, and same-machine intended-agent identity. The strongest legacy-design signals are request-gated acceptance, nonce/public-key binding, and a legacy-side relay / same-endpoint comparison under leakage, with `NO_TEE` fail-closed handling retained as a narrower implementation condition.

Legacy ProVerif Results

Model thread	Correspondence query	Result	Interpretation
Request-gated acceptance	`ClientAcceptsLegacy ==> ClientRequestsEvidence`	true	Within this abstraction, legacy acceptance still implies a prior evidence request.
Server-side attestation origin	`ClientAcceptsLegacy ==> ServerIssuesLegacyAttestation`	true	Within this abstraction, legacy acceptance still tracks a genuine server-side legacy attestation event.
Server-side binding/construction event	`ClientAcceptsLegacy ==> ServerCreatesLegacyReport`	true	Within this abstraction, legacy acceptance remains tied to the server-issued nonce/public-key report.
Relay / same-endpoint authenticity	`ClientAcceptsLegacy ==> LegacyServerBindsSameChannel`	false	Under the explicit leakage assumption, the legacy relay model does not establish same-endpoint authenticity for a client-accepted session.

Table 6: Summary of the legacy-design ProVerif correspondence queries used in this workspace.

Legacy Pre-Handshake Flow

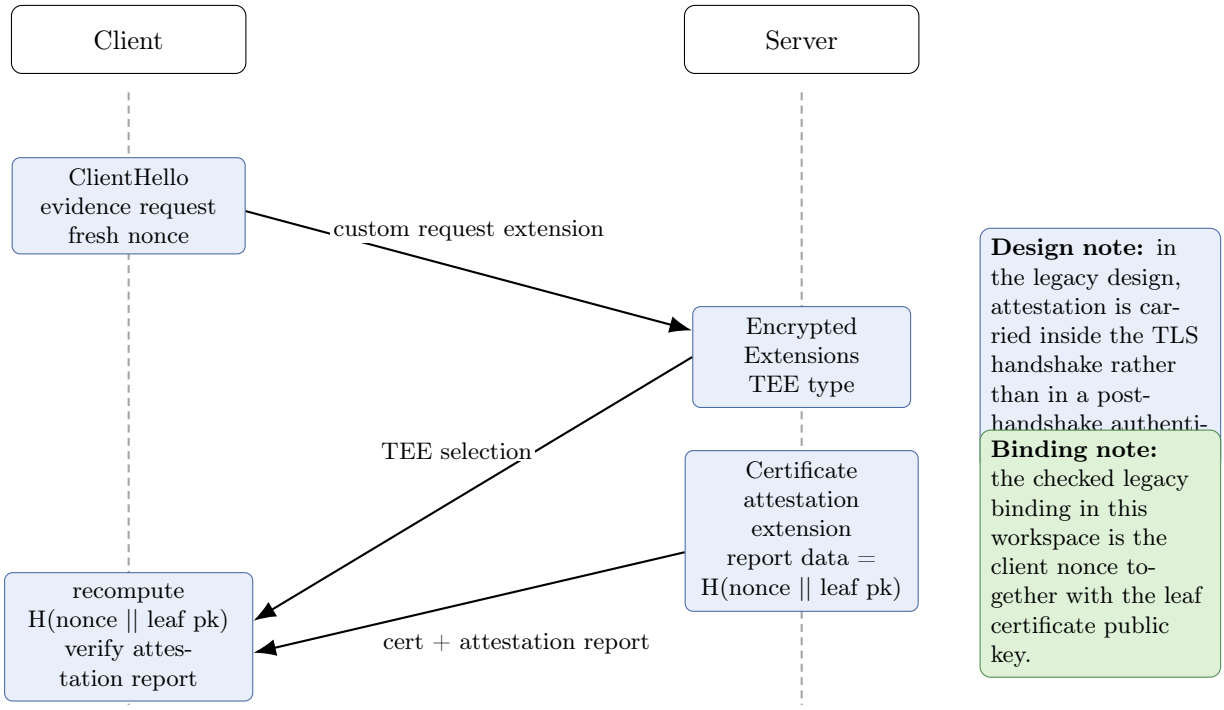

Figure 3: Legacy pre-handshake flow. This appendix figure is included only for comparison. Blue boxes show protocol steps, and the green note marks the explicit binding checked in the legacy artifacts.